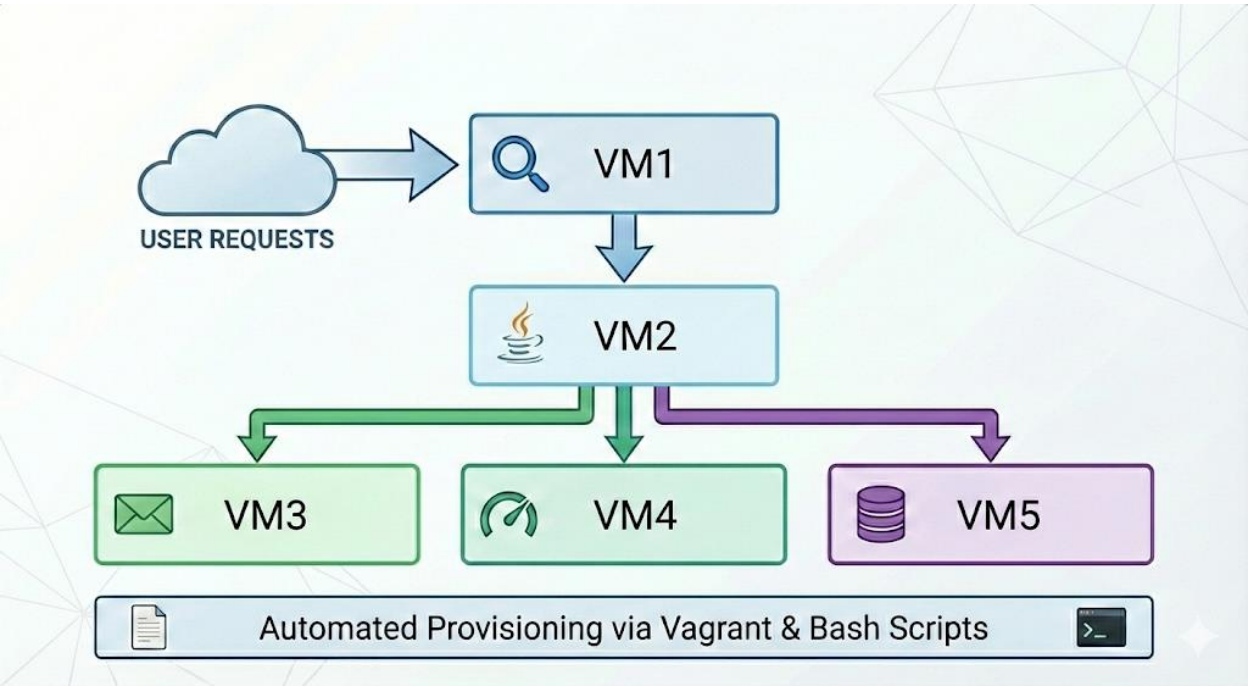
Automated Multi-Tier Web App Deployment Project



From Manual Chaos to One-Click Provisioning: Automating the Deployment of a Multi-Tier Java Stack (Nginx, Tomcat, RabbitMQ, Memcached, MySQL) using **Vagrant** & **Bash**.

Prepared by : Mohamed Ayman Nabawi

1. Executive Summary

This project focuses on deploying a complex, multi-tier Java web application that simulates a real-world social networking platform. The project was executed in two phases:

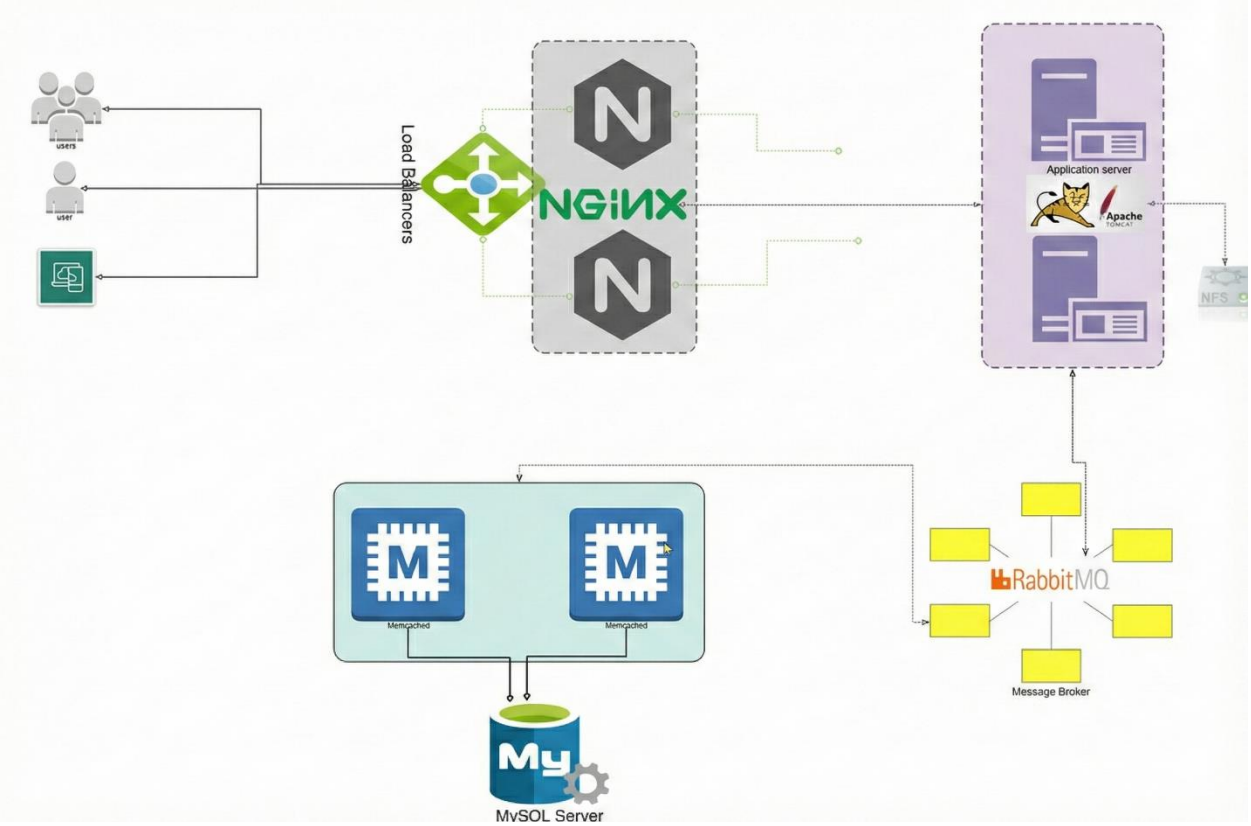
- **Manual Deployment:** To understand the intricacies of service dependencies and configuration.
- **Automated Provisioning:** Using **Bash Scripting** and **Vagrant** to convert the infrastructure into Code (IaC), reducing deployment time and eliminating human error.

2. System Architecture

The application relies on a microservices-style architecture where multiple services communicate to handle user requests, data storage, and caching.

The Stack Components:

- **Load Balancer:** Nginx (Handles HTTP requests and forwards them to the App Server).
- **Application Server:** Apache Tomcat (Runs the Java Spring Boot artifact).
- **Message Broker:** RabbitMQ (Handles asynchronous messaging).
- **Database Caching:** Memcached (Speeds up database queries).
- **Database:** MySQL/MariaDB (Stores user data).



Scenario: Request Flow

Here is how the application handles a user request efficiently:

- **Nginx (Load Balancer):** The entry point that receives user traffic and forwards it to the application server.

- **Tomcat (App Server):** It processes the logic but relies on two helpers for performance:
    - **Memcached:** Caches database queries for **speed**.
    - **RabbitMQ:** Queues messages for **stability** (Asynchronous processing).

- **MySQL (Database):** The final destination where data is permanently stored.

3. The Challenge: Manual Deployment

Initially, the stack was set up manually on virtual machines. This process revealed several critical challenges:

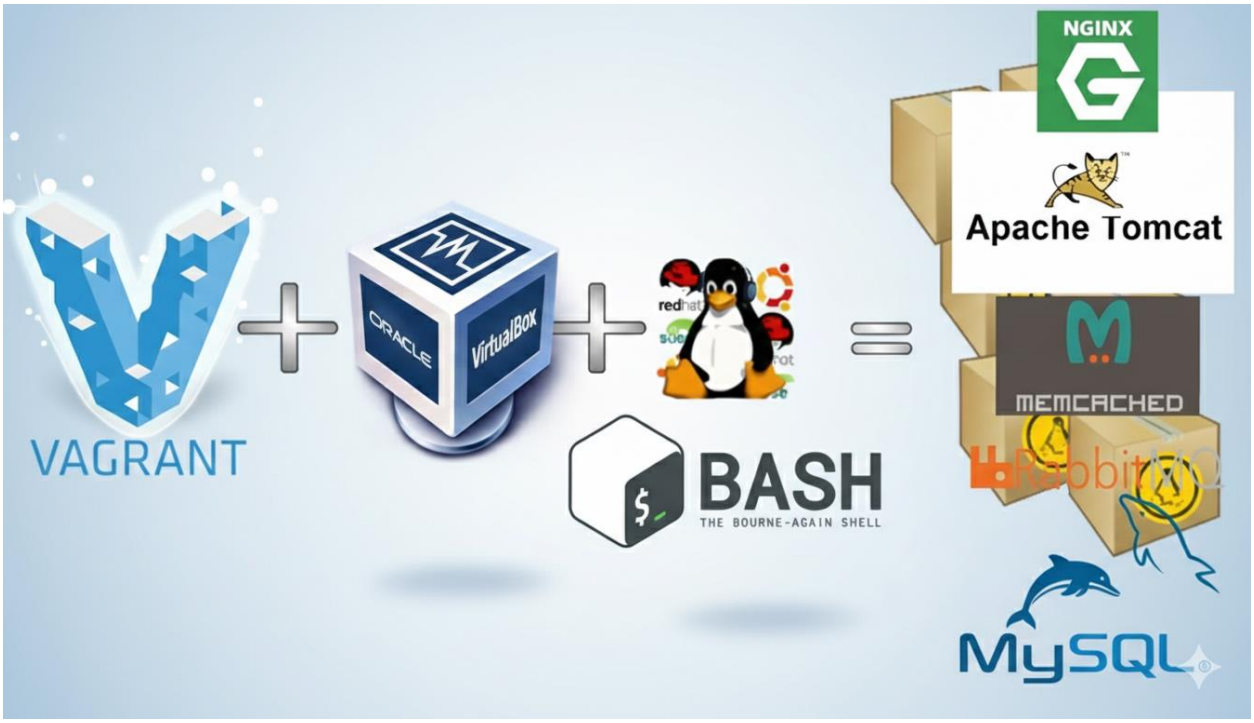
- **Time-Consuming:** Provisioning 5 VMs and installing dependencies took over 2 hours.
- **Complex Dependencies:** Services had to be started in a specific order (DB -> Memcached -> RabbitMQ -> Tomcat -> Nginx).
- **Human Error:** Manual configuration of IP addresses and firewalls often led to connectivity issues.
- **Lack of Portability:** The environment was difficult to replicate on other machines.

4. The Solution: Infrastructure as Code (Automation)

To overcome manual limitations, I automated the entire stack using **Vagrant** for VM management and **Bash Scripts** for provisioning.

Tools Used:

- **Hypervisor:** Oracle VirtualBox.
- **Automation:** Vagrant & Bash Scripting.
- **OS:** Linux (CentOS 7 for services, Ubuntu for Web).
- **Build Tool:** Maven.



5. Execution Flow: The Automation Roadmap

Here is the step-by-step journey of how the environment is built, from source code to a running application:

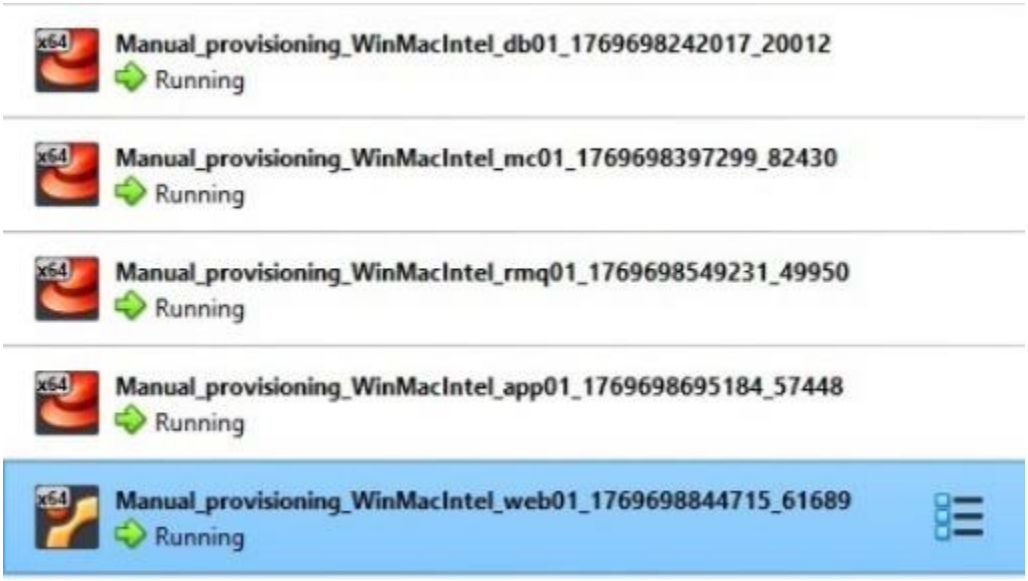
Phase 1: Preparation 🛠️

1. **Prerequisites:** Ensure the necessary tools (VirtualBox, Vagrant, and Git) are installed.
2. **Clone Repository:** Download the project source code using the command: git clone https://github.com/MoNabawy-2003/Automated-Multi-Tier-Web-App-Deployment.git

3. **Initialization:** Open the terminal (Git Bash) and navigate into the project directory containing the Vagrantfile.

Phase 2: The "One-Click" Action 🚀

4. **Launch Infrastructure:** Execute the main automation command: vagrant up *(This single command triggers the creation of all 5 Virtual Machines).*

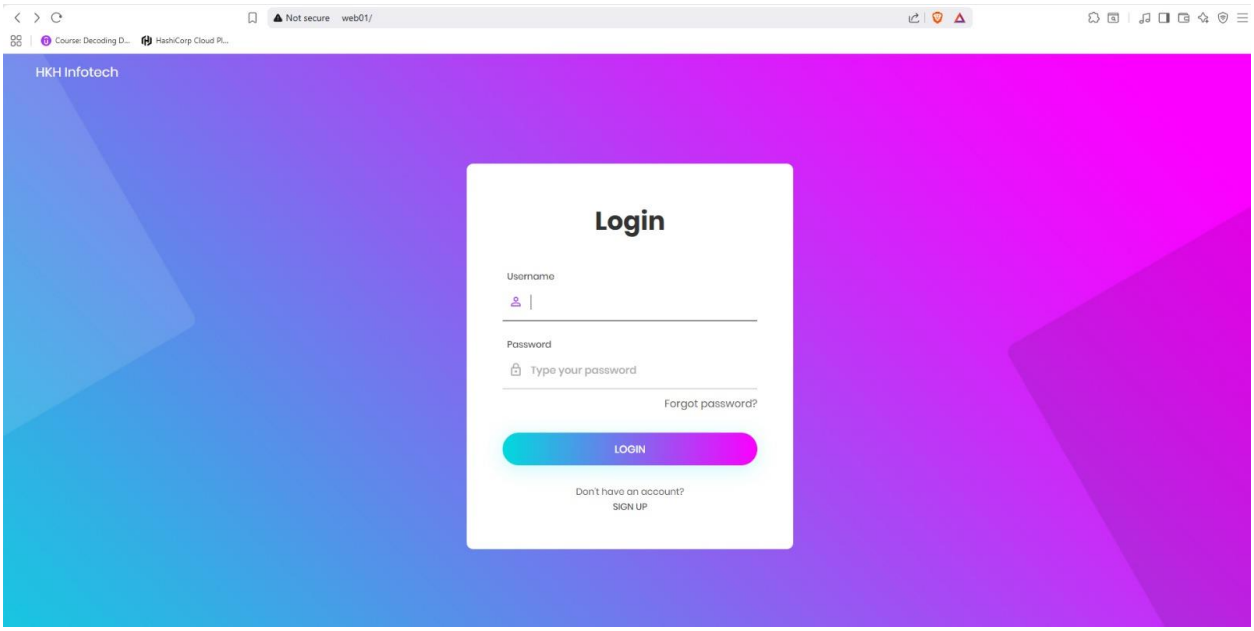


Phase 3: Automated Provisioning (Internal Logic) ⚙

5. **Service Configuration:** Once the VMs are up, Vagrant automatically executes the Bash scripts to set up the stack in the correct order:
    - **Data Layer:** Installs **MySQL**, configures the firewall, and restores the database dump.
    - **Cache & Queue:** Installs and starts **Memcached** and **RabbitMQ** services.
    - **App Layer:** Clones the source code, builds the artifact using **Maven**, and deploys it to **Tomcat**.
    - **Web Layer:** Configures **Nginx** as a Load Balancer/Reverse Proxy.

Phase 4: Verification ✅

6. **Final Validation:**
    - Check the status of the machines using vagrant status.
    - Open the browser and access the application URL to confirm the deployment is live and connected.



📂 Full Source Code & Scripts:

https://github.com/MoNabawy-2003/Automated-Multi-Tier-Web-App-Deployment.git